```
#from multipledispatch import dispatch

# addition
'''
@dispatch(int, int)

def add(a, b):
print(a + b)
def add(a, b):
print(a*b)
add(2, 3)
'''
# cancatenation
'''
@dispatch(str, str)

def concat(a, b):
print(a + b)
concat("Hello, ", "World!")
'''
# repeat string
'''
@dispatch(int,str)

def repeat_string(n, s):
print(s * n)
repeat_string(3, "Hi! ")
'''
# method overloading in a class -- compile time polymorphism
'''
class A:
@dispatch(str, str)
def add(self, a, b):
print(a + b)

@dispatch(int, int)
def add(self, a, b):
print(a + b)

@dispatch(int, str)
def add(self, a, b):
print(a*b)
a = A()
a.add(4, 5)
a.add("Hello, ", "World!")
a.add(3, "Hi! ")
'''


# method overriding - run time polymorphism

# Different class same method name and same parameters it suppoorts python
'''
class A:
```

```
def cash(self):
print("A")
class B(A):
def bike(self):
print("B")
b= B()
b.cash()
'''
# method overriding in normal class -- compile time polymorphism

# same class and same method name but different parameters.

## python doesn't support this  so we can achieve through importing multiple dispatch --
@dispatch
'''
class A:
def cash(self):
print("A")
class B(A):
def cash(self):
print("B")
b= B()
b.cash()
'''

# method overriding
'''
class A():
def display(self, m, n):
print(m + n)
class B(A):
def display(self, m, n):
print(m * n)
b = B()
b.display("Hello ", 3)
'''
```

```
#   ----------       Polymorphism in Python  ----------

# Polymorphism is one of the core pillars of Object-Oriented Programming (OOP).

# The word comes from Greek, meaning "poly-many morphism-forms". In programming,

# it allows a single interface or function to work with different types of data or objects.

#_____-_____

#Types of Polymorphism
```

mehod overloading - compile time polymorphism

This occurs when multiple methods in the same class have the same name but different parameters (different number or types of arguments).
The compiler determines which method to call based on the input.

same class and same method name but different parameters.

python doesn't support this so we can achieve through importing from multipledispatch import dispatch -- @dispatch

method overloading in a class -- compile time polymorphism
'''
class A:
@dispatch(str, str)
def add(self, a, b):
print(a + b)

@dispatch(int, int)
def add(self, a, b):
print(a + b)

@dispatch(int, str)
def add(self, a, b):
print(a*b)
a = A()
a.add(4, 5)
a.add("Hello, ", "World!")
a.add(3, "Hi! ")
'''

method overriding - run time polymorphism

This occurs when a child class provides a specific implementation of a method that is already defined in its parent class.
The method that gets executed is determined at "runtime" based on the object being used.

Different class same method name and same parameters it suppoorts python

method overriding in normal class
'''
class A:
def cash(self):
print("A")
class B(A):
def cash(self):
print("B")
b= B()
b.cash()
'''